

MACHINE LEARNING FUNDAMENTALS SERIES

Word Embedding & Word2Vec: Principles, Geometry & Optimization

Dense Vector Representations, CBOW, Skip-Gram, and Negative Sampling Optimization

1. Introduction and Problem Motivation

Machine Learning models and Neural Networks are fundamentally mathematical engines that operate on numbers, vectors, and matrices. They cannot natively process raw unstructured text strings. Converting human language into numerical formats is therefore a fundamental prerequisite for Natural Language Processing (NLP).

1.1 The Failure of Naive Random Number Assignment

An intuitive early approach was to assign a unique random scalar value to each vocabulary word (e.g., "great!" = 4.2, "awesome!" = -32.1, "horrible!" = 105.7).

- **Arbitrary Distance:** Synonyms such as "great!" and "awesome!" receive completely unrelated numerical values far apart on the number line.
- **Lack of Generalization:** A neural network cannot infer that two words sharing identical semantic meanings should trigger similar hidden activations or outputs.

1.2 The Core Objective of Word Embeddings

Word Embedding maps each vocabulary word to a D -dimensional continuous vector space ($\mathbb{R}^D$). The primary objective is to position semantically and contextually related words **close to each other in vector space**. Multi-dimensional vectors allow models to capture multiple nuanced semantic facets simultaneously (e.g., positive sentiment vs. sarcastic usage).

2. Neural Network Architecture for Learning Embeddings

Word embeddings are not assigned manually; they are learned automatically by training a simple feed-forward neural network on word co-occurrence patterns.

Consider a minimalist training corpus containing two sentences:

Sentence 1: "Troll 2 is great!" Sentence 2: "Gymkata is great!" (1)

Vocabulary $V = \{\text{"Troll 2", "Gymkata", "is", "great!"}\}$ with $|V| = 4$.

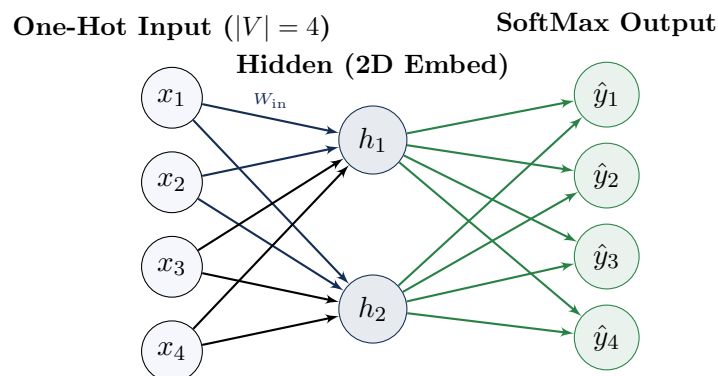


Figure 1: Minimalist Neural Network Architecture for Word Embedding Generation (2D Embedding Space).

2.1 Network Components and Functionality

1. **One-Hot Input Layer** ($|V| = 4$): Each word is represented as a 4-dimensional binary vector where only the index corresponding to the target word is 1, and all other entries are 0 (e.g., $\mathbf{x}_{\text{"Troll 2"}} = [1, 0, 0, 0]^T$).
2. **Hidden Layer** ($D = 2$): The number of hidden activation nodes equals the desired embedding dimension (2D for visualization). The activation function is the linear **Identity Function** ($f(x) = x$).
3. **Input-to-Hidden Weight Matrix** (W_{in}): Has shape $(|V| \times D) = (4 \times 2)$. **This matrix IS the Word Embedding table!** Each row i contains the 2D coordinate weights for vocabulary word i .
4. **Output Layer & SoftMax Loss**: Multiplies hidden activations by W_{out} (2×4), applies SoftMax to output probability distribution $\hat{\mathbf{y}}$, and computes Cross-Entropy Loss against the target context word.

3. Geometric Vector Space Mechanics during Training

- **Initial Un-trained State**: Before training, weights in W_{in} are initialized randomly. The coordinate vectors for *"Troll 2"* (e.g., $[0.38, 0.42]$) and *"Gymkata"* (e.g., $[-0.85, 0.12]$) are distant from each other in 2D space.
- **Backpropagation Gradient Updates**: As the network is trained to predict the succeeding word *"is"* given input *"Troll 2"*, Backpropagation computes gradients and shifts the row weights for *"Troll 2"*. Subsequently, when trained on *"Gymkata is"*, gradients shift the row weights for *"Gymkata"* in the exact same vector direction.
- **Final Convergence**: Because both *"Troll 2"* and *"Gymkata"* appear in identical context preceding *"is"*, Backpropagation systematically pulls their 2D vector coordinates together, **clustering them tightly in vector space**.

4. Word2Vec Framework: CBOW vs. Skip-Gram

The Word2Vec framework extends basic single-word prediction to broader context windows:

- **Continuous Bag-of-Words (CBOW)**: Uses surrounding context words to **predict the target center word** (e.g., given context words *"Troll 2"* and *"great!"*, predict center word *"is"*).
- **Skip-Gram**: Uses the target center word to **predict surrounding context words** (e.g., given center word *"is"*, predict context words *"Troll 2"*, *"great!"*, and *"Gymkata"*).

5. Scaling to Massive Vocabularies: Negative Sampling

5.1 The Computational Explosion Bottleneck

In production NLP applications (e.g., training on Wikipedia with $|V| = 3,000,000$ words and $D = 100$ embedding dimensions):

- Number of Input-to-Hidden weights: $3,000,000 \times 100 = 300,000,000$ parameters.
- Number of Hidden-to-Output weights: $100 \times 3,000,000 = 300,000,000$ parameters.
- **Total Parameters: 600,000,000 weights**. Computing SoftMax over 3 million output classes per step is computationally prohibitive.

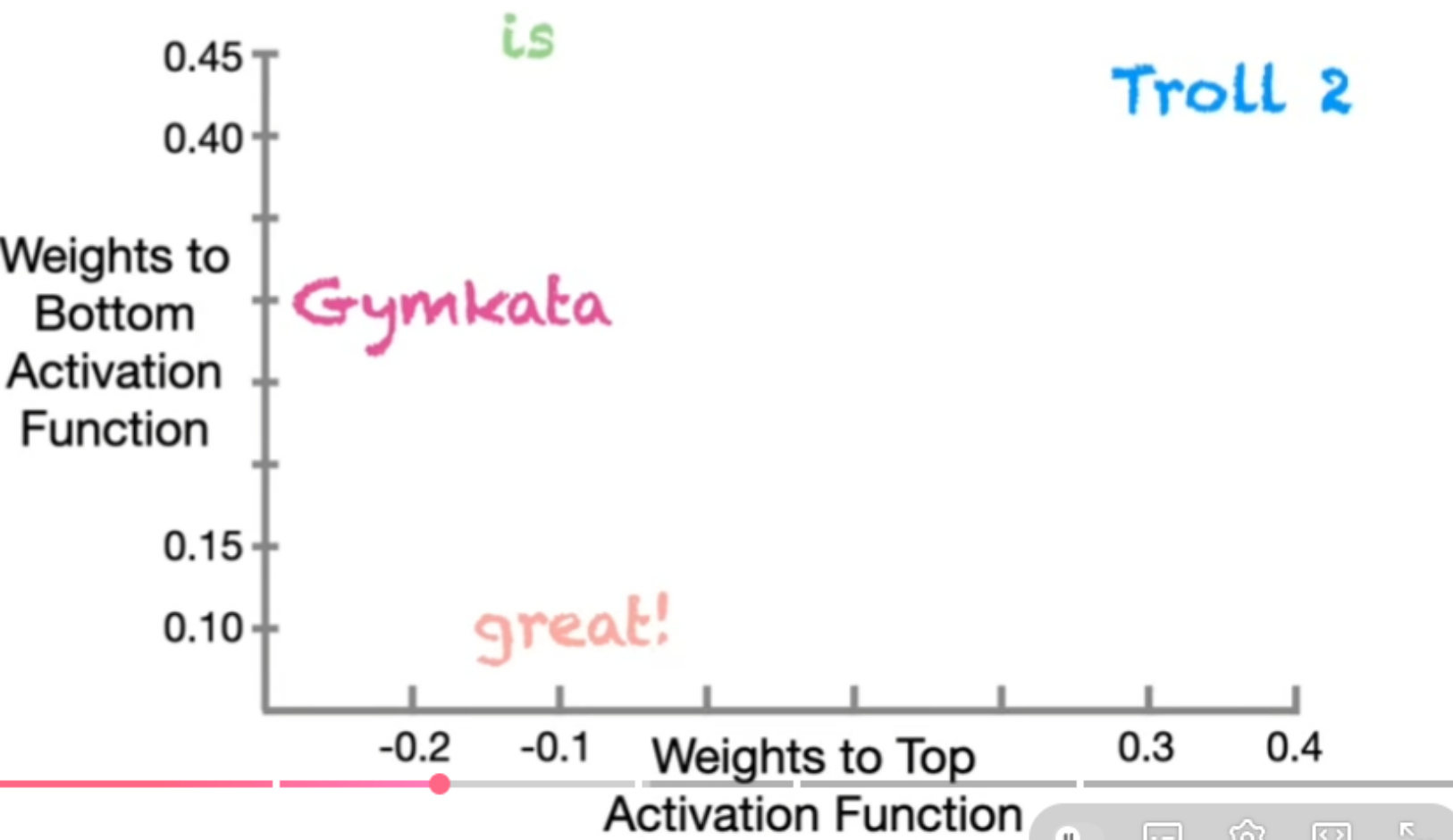
5.2 The Negative Sampling Optimization Strategy

Word2Vec resolves this bottleneck using **Negative Sampling**:

1. **Input Sparsity Exploitation:** Because the input vector is One-Hot, 2,999,999 zero entries multiply by zero, instantly bypassing 300,000,000 input weight computations.
2. **Output Subsampling:** Instead of evaluating SoftMax across all 3 million output words, Negative Sampling randomly selects a small sample of 2 to 20 incorrect **Negative Words** (e.g., “*abandon*”) along with 1 correct **Positive Target Word** (e.g., “*is*”).
3. **Drastic Weight Reduction:** Backpropagation updates only the weights connecting to the selected positive word and negative samples. The number of updated weights per iteration drops from 600,000,000 down to **approximately 300 weights**, enabling lightning-fast training on massive corpora.

6. Summary Comparison: Lecture Example vs. Production Word2Vec

Table 1: Comparative Overview: Pedagogical Example vs. Production Word2Vec.		
Feature / Aspect	Pedagogical Lecture Example	Production Word2Vec Implementation
Vocabulary Size ($ V $)	4 words (<i>Troll 2</i> , <i>Gymkata</i> , <i>is</i> , <i>great!</i>)	$\sim 3,000,000$ words (Google News / Wiki)
Embedding Dimensions (D)	2D (for x, y graph plotting)	100D to 300D dense vector space ($\mathbb{R}^{300}$)
Loss Function	Full 4-class SoftMax + Cross Entropy	**Negative Sampling** (2–20 negative words)
Active Weights / Step	16 parameters	~ 300 parameters (bypasses 600M weights)
Primary Objective	Visual demonstration of clustering	High-performance semantic vector space

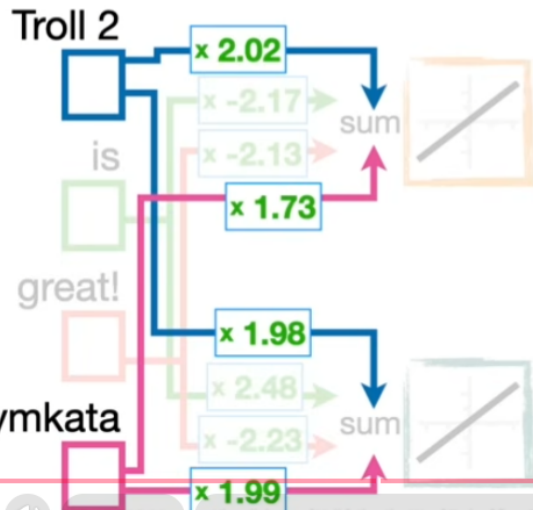




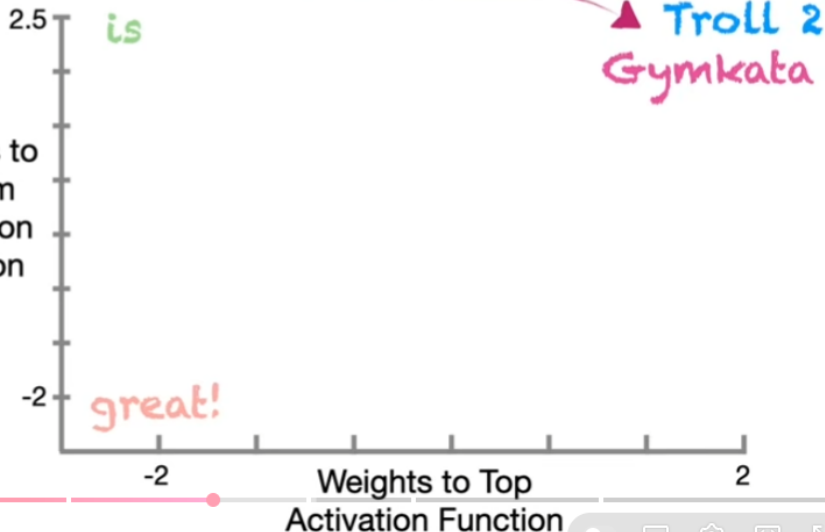
...we see that **Troll 2** and **Gymkata** are now relatively close to each other compared to the other words in the training data.

Training Data

Troll 2 is great!
Gymkata is great!



Weights to Bottom Activation Function

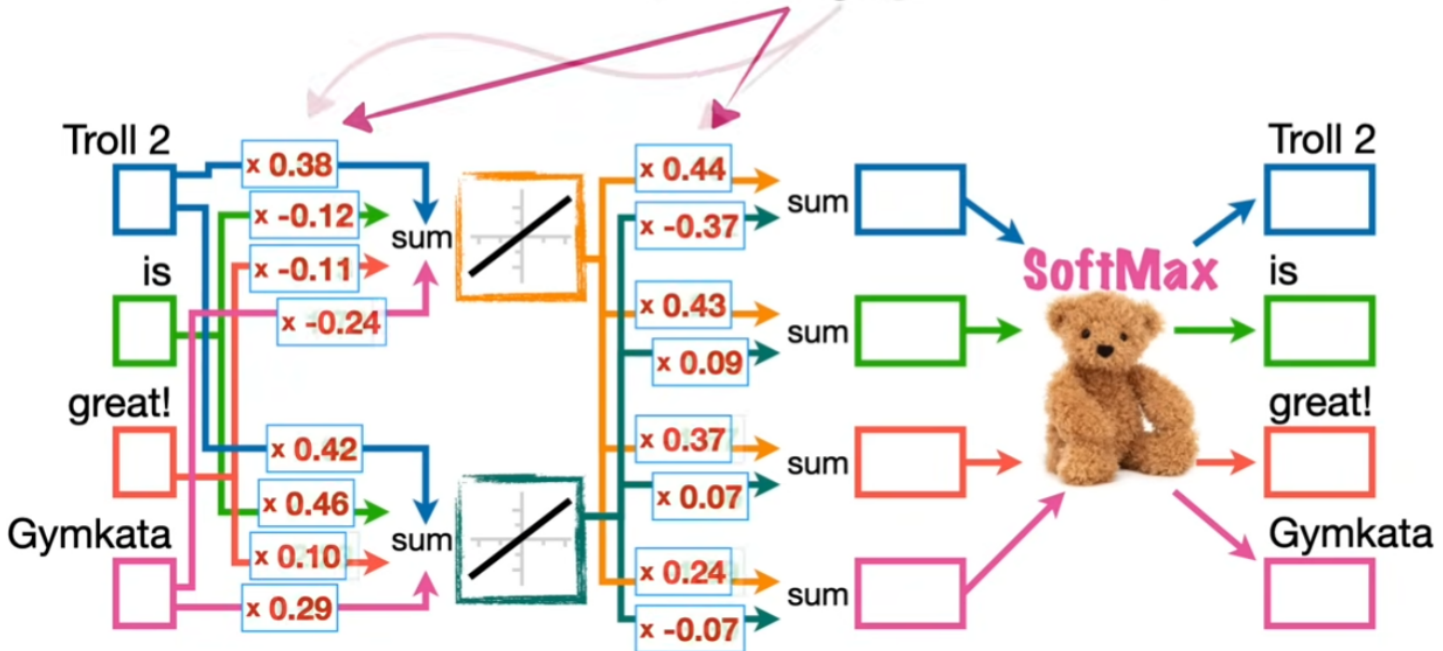




Training Data

Troll 2 is great!
Gymkata is great!

...So we start with
these **Weights**...





$$\begin{array}{r} 3,000,000 \\ \times \\ 100 \\ \times \\ 2 \\ \hline 600,000,000 \end{array}$$

...for a total of
600,000,000 Weights.

Training Data

The Aardvark is...
It is the only...
Unlike most...
...

